

Writing a Generalized Concurrent Queue

A two-lock queue with pretty good concurrency

By Herb Sutter

Herb is a software development consultant, a software architect at Microsoft, and chair of the ISO C++ Standards committee. He can be contacted at www.gotw.ca.

Last month [1], I showed code for a lock-free queue that supported the limited case of exactly two threads—one producer, and one consumer. That's useful, but maybe not as exciting now that our first rush of lock-free coding glee has worn off. This month, let's tackle the general problem of supporting multiple producers and multiple consumers with as much concurrency as possible. The code in this article uses four main design techniques:

First, we'll use (the equivalent of) two locks: One for the head end of the queue to regulate concurrent consumers, and one for the tail to regulate concurrent producers. We'll use ordered atomic variables (C++0x *atomic*<>, Java/.NET *volatile*) directly instead of prefabricated mutexes, but functionally we're still writing spinlocks; we're just writing them by hand. Although this means it's not a purely "lock-free" or nonblocking algorithm, it's still quite concurrent because we'll arrange the code to still let multiple consumers and multiple producers make progress at the same time by arranging to do as much work as possible outside the small critical code region that updates the head and tail, respectively.

Second, we'll have the nodes allocate the contained *T* object on the heap and hold it by pointer instead of by value. [2] To experienced parallel programmers this might seem like a bad idea at first, because it means that when we allocate each node we'll also need to perform an extra heap allocation, and heap allocations are notorious scalability busters on many of today's memory allocators. It turns out that, even on a system with a nonscalable allocator, the benefits typically outweigh the advantages: Holding the *T* object by pointer let us get greater concurrency and scalability among the consumer threads, because we can take the work of actually copying the *T* value out of the critical section of code that updates the shared data structure.

Third, we don't want to have the producer be responsible for lazily removing the nodes consumed since the last call to *Produce*, because this is bad for performance: It adds contention on the queue's head end, and it needlessly delays reclaiming consumed nodes. Instead, we'll let each consumer be responsible for trimming the node it consumed, which it was touching anyway and so gives better locality.

Fourth, we want to follow the advice that "if variables *A* and *B* are not protected by the same mutex and are liable to be used by two different threads, keep them on separate cache lines" to avoid false sharing or "ping-ponging" which limits scalability. [3] In this case, we want to add padding to ensure that different nodes (notably the first and last nodes), the *first* and *last* pointers into the list, and the *producerLock* and *consumerLock* variables are all on different cache lines.

A Two-Lock Multiproducer/Consumer Queue

The queue data structure itself is a singly linked list. To make the code simpler for the empty-queue boundary case, the list always contains a dummy node at the head, and so the *first* element logically in the queue is the one contained in the node after first. Figure 1 shows the layout of an empty queue, and Figure 2 shows a queue that holds some objects.

Each node holds the *T* object by pointer and adds padding:

```
template <typename T>
struct LowLockQueue {
private:
    struct Node {
```

```
Node( T* val ) : value(val), next(nullptr) { }
T* value;
atomic<Node*> next;
char pad[CACHE_LINE_SIZE - sizeof(T*) - sizeof(atomic<Node*>)];
};
```

Like any shared variable, the *next* pointer needs to be protected by a mutex or, as here, declared as an ordered atomic type (C++0x *atomic<>* or Java/.NET *volatile*). The padding here is to ensure that two *Node* objects won't occupy the same cache line; in particular, in a nonempty queue, having the first and last nodes in the same cache line would penalize performance by causing invisible contention between the producer and consumer. Note that the amount of padding shown here and later on errs on the side of being too conservative: Each *Node* will be allocated on the heap, and a heap allocator may add its own extra overhead to each allocated block for alignment and housekeeping information, which effectively adds extra padding. If so, and if we know how much that will be, we could reduce our internal padding proportionately to make a heap-allocated *Node* exactly fill one cache line.

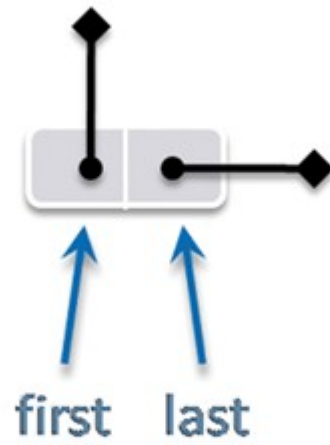


Figure 1: Structure of an empty queue.

[\[Click image to view at full size\]](#)

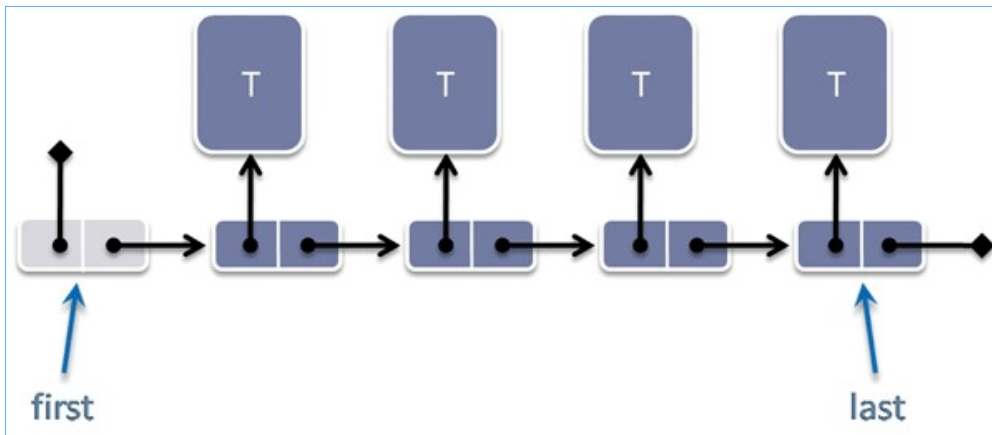


Figure 2: A queue containing four T objects

Next, here are our queue control variables:

```
char pad0[CACHE_LINE_SIZE];

// for one consumer at a time
Node* first;

char pad1[CACHE_LINE_SIZE
          - sizeof(Node*)];

// shared among consumers
atomic<bool> consumerLock;

char pad2[CACHE_LINE_SIZE
          - sizeof(atomic<bool>)];

// for one producer at a time
Node* last;

char pad3[CACHE_LINE_SIZE
          - sizeof(Node*)];

// shared among producers
atomic<bool> producerLock;

char pad4[CACHE_LINE_SIZE
          - sizeof(atomic<bool>)];
```

Again, we add padding to make sure that data used by different threads stay physically separate in memory and cache. Clearly, we want the consumer-end data and the producer-end data to be on separate cache lines; but even though only one producer and one consumer will be active at a time, we want to keep the locking variable separate so that waiting consumers spinning on *consumerLock* won't contend on the cache line that contains *first* which the ac

consumer is updating, and that waiting producers spinning on *producerLock* won't slow down the active producer who is updating *last*.

The constructor just sets up the initial empty state, and the destructor (in .NET or Java, this would be the disposer) just walks the internal list and tears it down:

```
public:
    LowLockQueue() {
        first = last = new Node( nullptr );
        producerLock = consumerLock = false;
    }
    ~LowLockQueue() {
        while( first != nullptr ) {                // release the list
            Node* tmp = first;
            first = tmp->next;
            delete tmp->value;                        // no-op if null
            delete tmp;
        }
    }
}
```

Produce

Now let's look at the first of the two key methods: *Produce*. The goal is to allow multiple producers, and to let them run as concurrently as possible:

```
void Produce( const T& t ) {
    Node* tmp = new Node( new T(t) );
    while( producerLock.exchange(true) )
        { } // acquire exclusivity
    last->next = tmp; // publish to consumer
    last = tmp; // swing last forward
    producerLock = false; // release exclusivity
}
```

First, we want to do as much work as possible outside the critical section of code that actually updates the queue. In this case, we can do all of the allocation and construction of the new node and its value concurrently with any number of other producers and consumers.

Second, we "commit" the change by getting exclusive access to the tail of the queue. The *while* loop keeps trying to set the *producerLock* to true until the old value was false because while the old value was true, it means someone else already has exclusivity. The way to read this *while* loop is, "until I get to be the one to change *producerLock* from false to true," which means that this thread has acquired exclusivity. Then we can update *last->next* and *last* itself, which are two separate writes and cannot be done as a single atomic operation on most processors without some sort of lock. Finally, we release exclusivity on the tail of the queue by setting *producerLock* to false.

Consume

Likewise, we want to support any number of threads calling *Consume*, and let them run as concurrently as possible. First, we get exclusivity, this time on the head end of the queue:

```
bool Consume( T& result ) {
    while( consumerLock.exchange(true) )
        { }           // acquire exclusivity
}
```

Next, we read the head node's *next* pointer. If it's not null, we need to take out the first value but we want to do as little work as possible here inside the exclusive critical section:

```
Node* theFirst = first;
Node* theNext = first->next;
if( theNext != nullptr ) {           // if queue is nonempty
    T* val = theNext->value;           // take it out
    theNext->value = nullptr;          // of the Node
    first = theNext;                  // swing first forward
    consumerLock = false;             // release exclusivity
}
```

Now we're done touching the list, and other consumers can make progress while we do the remaining copying and cleanup work off to the side:

```
result = *val;           // now copy it back
delete val;              // clean up the value
delete theFirst;         // and the old dummy
return true;             // and report success
}
```

Otherwise, if *theNext* was null, the list was empty and we can immediately release exclusion and return that status:

```
consumerLock = false;           // release exclusivity
return false;                   // report queue was empty
}
};
```

Fully Nonblocking Multiproducer/Consumer Queues

The above code still uses two locks, albeit sparingly. How might we eliminate the producer and consumer locks for a fully nonblocking queue implementation that allows multiple concurrent producers and consumers? If that intrigues you, and you're up for some down-and-dirty details, here are two key papers you'll be interested in reading.

In 1996, Michael and Scott published a paper that presented two alternatives for writing an internally synchronized queue. [4] One alternative really is nonblocking; the other uses a producer lock and a consumer lock, much like the examples in this article. In 2003, Herlihy, Luchango and Moir pointed out scalability limitations in Michael and Scott's approach, and presented their own obstruction-free queue implementation. [5]

Both of these papers featured approaches that require a double-width compare-and-swap operations (also known as "DCAS") that can treat a pointer plus an integer counter together as a single atomic unit. That is problematic because not all platforms have a DCAS operation, especially mainstream

processors in 64-bit mode which would essentially require a 128-bit CAS. [6] One also requires a special free list allocator to work properly.

Coming Up

We applied four techniques:

1. Having two locks, one for each end of the queue.
2. Allocating objects on the heap to let us make consumers more concurrent.
3. Having consumers remove consumed nodes one at a time for better locality, less contention at the head, and more immediate cleanup than having producers lazily clean up consumed nodes.
4. Adding padding to keep data used by different threads on different cache lines, avoiding memory performance penalties due to false sharing or "ping-pong."

But just how much did each of those help, and how much did each help depending on the size of the queued objects? Next month, I'll break down the four techniques by analyzing the successive performance impact of each of these techniques with some pretty graphs. Stay tuned.

Notes

[1] H. Sutter. "Lock-Free Code: A False Sense of Security" (*DDJ*, June 2008). Available online at <http://ddj.com/architect/208200273>.

[2] Note that this happens naturally in Java and .NET for reference types, which are always held indirectly via a pointer (which is called an object reference in those environments).

[3] H. Sutter. "Maximize Locality, Minimize Contention" (*DDJ*, September 2008). Available online at <http://ddj.com/architect/208200273>.

[4] M. Michael and M. Scott. "Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms" (*Proceedings of the 15th ACM Symposium on Principles of Distributed Computing*, 1996)

[5] M. Herlihy, V. Luchango and M. Moir. "Obstruction-Free Synchronization: Double-Ended Queues As an Example" (*Proceedings of the 23rd International Conference on Distributed Computing Systems*, 2003).

[6] You could try to make it work in 64 bits via heroic efforts to steal from the 64-bit address space, taking ruthless advantage of the knowledge that on mainstream systems today the operating system usually doesn't actually use all 64 bits of addresses and might not notice if you use a few or even a dozen for your own ends. However, that's inherently brittle and nonportable, and a lot of other people (including probably your OS's developers) have had the same idea and tried to grab those bits too in the frenzied 64-bit land rush already in progress. In reality, you generally only get to play this kind of trick if you're the operating system or its close friend.